

Page Object Model (POM)

1 – Introdução e Estrutura Básica

Objetivo

Entender o que é o Page Object Model (POM) e por que ele é importante para organizar e manter os testes automatizados.

Aprender como criar a estrutura básica do POM e implementar uma classe base com métodos comuns.

Conceito

O POM é um padrão de projeto que representa cada página da aplicação como uma classe. Essa classe agrupa os seletores e as ações que podem ser feitas naquela página. Isso ajuda a deixar os testes mais organizados e fáceis de manter.

A estrutura básica inclui uma pasta para as páginas (como `page_objects`) e uma classe base (`BasePage`) com métodos genéricos que serão usados pelas páginas específicas.

Exemplo da classe base (`base_page.py`):

```
class BasePage:
    def __init__(self, page):
        self.page = page
        self.botao_home = page.get_by_role("link", name="Home")
        self.botao_produtos = page.get_by_role("link", name="Products")
        self.botao_carrinho = page.get_by_role("link", name="Cart")
        self.botao_cadastro_login = page.get_by_role("link", name="Signup / Login")

    def acessar_home(self):
        self.page.goto('https://automationexercise.com/')
```

```
def acessar_carrinho(self):
    self.page.goto('https://automationexercise.com/view_cart')

def acessar_cadastro_login(self):
    self.page.goto('https://automationexercise.com/login')
```

2 – Criando uma Página Específica: LoginPage

Objetivo

Aprender a criar uma classe de página específica que encapsula os seletores e as ações da página de login.

Como funciona

A classe `CadastroLogin` herda da `BasePage`. Ela define os seletores da página como atributos e implementa métodos para navegar até a página e realizar o login.

Isso torna o código dos testes mais limpo, pois o teste chama esses métodos, sem precisar se preocupar com detalhes dos seletores.

Exemplo de `cadastro_login.py` :

```
from pages.base_page import BasePage

class CadastroLogin(BasePage):

    def __init__(self, page):
        super().__init__(page)
        self.input_email_login = page.locator("form").filter(has_text="Login").get_by_placeholder("Email Address")
        self.input_senha_login = page.get_by_role("textbox", name="Password")
        self.botao_login = page.get_by_role("button", name="Login")

    def fazer_login(self, email="", senha=""):
        self.input_email_login.fill(email)
```

```
self.input_senha_login.fill(senha)
self.botao_login.click()
```

3 – Escrevendo Testes Usando o POM

Objetivo

Aprender a usar as classes de página para escrever testes simples, claros e fáceis de manter.

Como funciona

Nos testes, você cria uma instância da página e chama seus métodos para realizar ações, sem lidar diretamente com seletores ou detalhes de interface.

Isso torna os testes mais legíveis e facilita a manutenção e reutilização de código.

Exemplo de teste `test_login.py` :

```
from pages.cadastro_login import CadastroLogin
from playwright.sync_api import expect

def teste_login_valido(page):
    login = CadastroLogin(page)
    login.acessar_cadastro_login()
    login.fazer_login(email='teste@testeabcde.com', senha='123456789')
    expect(page.get_by_role("link", name="Logout")).to_be_visible()
    page.pause()
```

4 – Boas Práticas e Dicas para POM

Objetivo

Conhecer boas práticas para aplicar o POM de forma eficiente e escalável.

Boas práticas

- Centralize os seletores dentro das classes de página.
- Use nomes claros e descritivos para métodos.

- Evite colocar lógica complexa dentro das páginas, mantenha nos testes ou helpers.
- Utilize herança para compartilhar funcionalidades comuns.

Referência oficial:

 <https://playwright.dev/python/docs/pom>